

Cài Java 17 (Amazon Corretto)

Cập nhật package

```
ec2-user@ip-172-31-10-181:~$ sudo dnf update --releasever=2023.10.20240105

A newer release of "Amazon Linux" is available.

Available Versions:
Version 2023.10.20240105:
Run the following command to upgrade to 2023.10.20240105:
dnf upgrade --releasever=2023.10.20240105

Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.10.20240105.html

Dependencies resolved.
=====
Package      Architecture Version      Repository    Size
-----
Upgrading:
nvml         x86_64      2025.06.11-1  cuda-rhel9-x86_64  8.9 M
Transaction Summary
-----
Upgrade 1 Package

Total download size: 8.9 M
Downloading Packages:
nvml-2025.06.11-1.x86_64.rpm 4.2 MB/s | 8.9 MB 00:02
Total
-----
4.2 MB/s | 8.9 MB 00:02

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing  : 1/1
Upgrading : nvml-2025.06.11-1.x86_64 1/2
Cleanup   : nvml-2025.06.10-1.x86_64 2/2
Verifying : nvml-2025.06.11-1.x86_64 1/2
Verifying : nvml-2025.06.10-1.x86_64 2/2

=====
WARNING:
A newer release of "Amazon Linux" is available.

Available Versions:
Version 2023.10.20240105:
Run the following command to upgrade to 2023.10.20240105:
dnf upgrade --releasever=2023.10.20240105

Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.10.20240105.html

=====

Upgraded:
nvml-2025.06.11-1.x86_64

Complete!
[ec2-user@ip-172-31-10-181 ~]$
```

Cài Java 17 Amazon Corretto

```
ec2-user@ip-172-31-10-181:~$ sudo dnf update --releasever=2023.10.20240105

Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.10.20240105.html

Dependencies resolved.
=====
Package      Architecture Version      Repository    Size
-----
Upgrading:
nvml         x86_64      2025.06.11-1  cuda-rhel9-x86_64  8.9 M
Transaction Summary
-----
Upgrade 1 Package

Total download size: 8.9 M
Downloading Packages:
nvml-2025.06.11-1.x86_64.rpm 4.2 MB/s | 8.9 MB 00:02
Total
-----
4.2 MB/s | 8.9 MB 00:02

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing  : 1/1
Upgrading : nvml-2025.06.11-1.x86_64 1/2
Cleanup   : nvml-2025.06.10-1.x86_64 2/2
Verifying : nvml-2025.06.11-1.x86_64 1/2
Verifying : nvml-2025.06.10-1.x86_64 2/2

=====
WARNING:
A newer release of "Amazon Linux" is available.

Available Versions:
Version 2023.10.20240105:
Run the following command to upgrade to 2023.10.20240105:
dnf upgrade --releasever=2023.10.20240105

Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.10.20240105.html

=====

Upgraded:
nvml-2025.06.11-1.x86_64

Complete!
ec2-user@ip-172-31-10-181:~$ sudo amazon-linux-extras enable corretto17
ec2-user@ip-172-31-10-181:~$ sudo yum install -y java-17-amazon-corretto-devel
ec2-user@ip-172-31-10-181:~$ sudo amazon-linux-extras install java-17-amazon-corretto-devel
Last metadata expiration check: 1 day, 14:40:36 ago on Sun Jan 19 23:44:32 2025.
Package java-17-amazon-corretto-devel-1:17.0.17-10-1.amzn2023.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-10-181 ~]$
```

Kiểm tra java

```
ec2-user@ip-172-31-10-181 ~]$ java -version
penjdk version "17.0.17" 2025-10-21 LTS
penJDK Runtime Environment Corretto-17.0.17.10.1 (build 17.0.17+10-LTS)
penJDK 64-Bit Server VM Corretto-17.0.17.10.1 (build 17.0.17+10-LTS, mixed mode, sharing)
ec2-user@ip-172-31-10-181 ~]$
```

Thiết lập JAVA_HOME

```
[ec2-user@ip-172-31-10-181 ~]$ echo "export JAVA_HOME=/usr/lib/jvm/java-17-amazon-corretto.x86_64" | sudo tee -a /etc/profile
echo "export PATH=\$JAVA_HOME/bin:\$PATH" | sudo tee -a /etc/profile
source /etc/profile
export JAVA_HOME=/usr/lib/jvm/java-17-amazon-corretto.x86_64
export PATH=\$JAVA_HOME/bin:\$PATH
[ec2-user@ip-172-31-10-181 ~]$
```

Kiểm tra alternatives

```
[ec2-user@ip-172-31-10-181 ~]$ sudo alternatives --config java
There is 1 program that provides 'java'.

   Selection    Command
-----
*+ 1            /usr/lib/jvm/java-17-amazon-corretto.x86_64/bin/java

Enter to keep the current selection[+], or type selection number:
```

Tải Kafka (KRaft mode)

Di chuyển tới thư mục cài đặt

cd /usr/local

```
[ec2-user@ip-172-31-10-181 ~]$ cd /usr/local
[ec2-user@ip-172-31-10-181 local]$
```

Tải Kafka mới (ví dụ 4.1.0)

sudo wget https://downloads.apache.org/kafka/4.1.0/kafka_2.13-4.1.0.tgz

```
[ec2-user@ip-172-31-10-181 local]$ sudo wget https://downloads.apache.org/kafka/4.1.0/kafka_2.13-4.1.0.tgz
--2026-01-20 14:30:29-- https://downloads.apache.org/kafka/4.1.0/kafka_2.13-4.1.0.tgz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 135.181.214.104, 2a01:4f8:10a:39da::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 133312825 (127M) [application/x-gzip]
Saving to: 'kafka_2.13-4.1.0.tgz.1'

kafka_2.13-4.1.0.tgz 100%[=====>] 127.14M  9.06MB/s  in 17s

2026-01-20 14:30:48 (7.44 MB/s) - 'kafka_2.13-4.1.0.tgz.1' saved [133312825/133312825]

[ec2-user@ip-172-31-10-181 local]$
```

Giải nén

sudo tar -xzf kafka_2.13-4.1.0.tgz

```
[ec2-user@ip-172-31-10-181 local]$ sudo tar -xzf kafka_2.13-4.1.0.tgz
[ec2-user@ip-172-31-10-181 local]$
```

Đổi tên thư mục cho gọn

sudo mv kafka_2.13-4.1.0 kafka

```
[ec2-user@ip-172-31-10-181 local]$ sudo mv kafka_2.13-4.1.0 kafka
[ec2-user@ip-172-31-10-181 local]$
```

Thiết lập quyền

sudo chown -R ec2-user:ec2-user /usr/local/kafka

```
[ec2-user@ip-172-31-10-181 local]$ sudo chown -R ec2-user:ec2-user /usr/local/kafka
[ec2-user@ip-172-31-10-181 local]$
```

Tạo thư mục lưu trữ dữ liệu và logs

sudo mkdir -p /usr/local/kafka/data

sudo mkdir -p /usr/local/kafka/logs

sudo chown -R ec2-user:ec2-user /usr/local/kafka/data /usr/local/kafka/logs

```
[ec2-user@ip-172-31-10-181 local]$ sudo mkdir -p /usr/local/kafka/data
sudo mkdir -p /usr/local/kafka/logs
sudo chown -R ec2-user:ec2-user /usr/local/kafka/data /usr/local/kafka/logs
[ec2-user@ip-172-31-10-181 local]$
```

Cấu hình Kafka KRaft Single-Node: Mở file và copy phần **tô vàng** vào

sudo nano /usr/local/kafka/config/server.properties

```
ec2-user@ip-172-31-10-181:/usr/local$ nano /usr/local/kafka/config/server.properties
# Licensed to the Apache Software Foundation (ASF) under one or more
# contributor license agreements.  See the NOTICE file distributed with
# this work for additional information regarding copyright ownership.
# The ASF licenses this file to You under the Apache License, Version 2.0
# (the "License"); you may not use this file except in compliance with
# the license.  You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the license is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the license for the specific language governing permissions and
# limitations under the license.
#
# ===== Server Basics =====
# The role of this server. Setting this puts us in KRaft mode
process.roles=broker,controller
# The node id associated with this instance's roles
node.id=1
# List of controller endpoints used connect to the controller cluster
controller.quorum.bootstrap.servers=localhost:9093
#
# ===== Socket Server Settings =====
# The address the socket server listens on.
# Combined mode (i.e. those with 'process.roles=broker,controller') must list the controller listener here at a minimum.
# If the broker listener is not defined, the default listener will use a host name that is equal to the value of java.net.InetAddress.getCanonicalHostName(),
# with PLAINTEXT listener name, and port 9092.
#
#   PLAINTEXT
#   listeners = listener_name://host_name:port
#
#   SASL
#   listeners = PLAINTEXT://your.host.name:9092
#   listeners=PLAINTEXT://:9092,CONTROLLER://:9093
#
# Name of listener used for communication between brokers.
listener.broker.listener.name=PLAINTEXT
#
# Listener name, hostname and port the broker or the controller will advertise to clients.
# If not set, it uses the value for 'listeners'.
advertised.listeners=PLAINTEXT://localhost:9092,CONTROLLER://localhost:9093
#
# A comma-separated list of the names of the listeners used by the controller.
# If no explicit mapping set in 'listener.security.protocol.map', default will be using PLAINTEXT protocol
# This is required if running in KRaft mode.
controller.listener.names=CONTROLLER
#
# Map listener names to security protocols, the default is for them to be the same. See the config documentation for more details
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,SASL:SASL,PLAINTEXT:SASL,SASL:SASL,SASL:SASL
#
# The number of threads that the server uses for receiving requests from the network and sending responses to the network
num.network.threads=3
#
# The number of threads that the server uses for processing requests, which may include disk I/O
num.io.threads=8

Help      Write Out  Where Is   Cut        Execute    Location   Read 100 lines  Undo      Set Mark   To Bracket  Previous   Back       Prev Word  Home
Exit      Read File  Replace   Paste     Justify    Go To Line  Redo      Copy       Where Was  Next       Forward    Next Word  End
```

```
GNU nano 2.9.3 /usr/local/kafka/config/server.properties
# 1. Durability: Unflushed data may be lost if you are not using replication.
# 2. Latency: Very large flush intervals may lead to latency spikes when the flush does occur as there will be a lot of data to flush.
# 3. Throughput: The flush is generally the most expensive operation, and a small flush interval may lead to excessive seeks.
# The settings below allow one to configure the flush policy to flush data after a period of time or
# every N messages (or both). This can be done globally and overridden on a per-topic basis.
#
# The number of messages to accept before forcing a flush of data to disk
log.flush.interval.messages=10000
#
# The maximum amount of time a message can sit in a log before we force a flush
log.flush.interval.ms=1000

##### Log Retention Policy #####
#
# The following configurations control the disposal of log segments. The policy can
# be set to delete segments after a period of time, or after a given size has accumulated.
# A segment will be deleted whenever "either" of these criteria are met. Deletion always happens
# from the end of the log.
#
# The minimum age of a log file to be eligible for deletion due to age
log.retention.hours=168
#
# A size-based retention policy for logs. Segments are pruned from the log unless the remaining
# segments drop below log.retention.bytes. Functions independently of log.retention.hours.
log.retention.bytes=1073741824
#
# The maximum size of a log segment file. When this size is reached a new log segment will be created.
log.segment.bytes=1073741824
#
# The interval at which log segments are checked to see if they can be deleted according
# to the retention policies
log.retention.check.interval.ms=300000
#####
# Kafka KRaft Mode Single-Node Configuration
#####
# ----- Mode & Cluster -----
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@3.104.237.133:9093
# cluster.id is set during start up using the format
#
# ----- Listeners -----
listeners=PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
advertised.listeners=PLAINTEXT://3.104.237.133:9092
controller.listener.names=CONTROLLER
#
# ----- Log / Storage -----
log.dirs=/usr/local/kafka/data
num.partitions=1
#
# ----- Kafka Internal Settings -----
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
default.replication.factor=1
min.insync.replicas=1

Help      Write Out  Where Is  Cut       Execute   Location  Undo      Set Mark  To Bracket Previous  Back      Prev Word Home
Exit      Read File  Replace   Paste     Justify   Go To Line Redo      Copy      Where Was Next      Forward  Next Word End
```

```
lcz-user@lcz-172-31-10-181 ~$ sudo nano /usr/local/kafka/config/server.properties
GNU nano 2.9.3 /usr/local/kafka/config/server.properties Modified
#####
# Kafka KRaft Mode Single-Node Configuration
#####
# ----- Mode & Cluster -----
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@3.104.237.133:9093
#
# ----- Listeners -----
listeners=PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
advertised.listeners=PLAINTEXT://3.104.237.133:9092
controller.listener.names=CONTROLLER
#
# ----- Log / Storage -----
log.dirs=/usr/local/kafka/data
num.partitions=1
#
# ----- Kafka Internal Settings -----
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
default.replication.factor=1
min.insync.replicas=1
```

Tạo systemd service cho Kafka: Mở file và copy phần tô vàng vào

sudo nano /etc/systemd/system/kafka.service

```
GNU nano 2.9.3 /etc/systemd/system/kafka.service
[Unit]
Description=Apache Kafka Server
After=network.target

[Service]
Type=simple
User=kafka-user
Group=kafka-user
ExecStart=/usr/local/kafka/bin/kafka-server-start.sh /usr/local/kafka/config/server.properties
ExecStop=/usr/local/kafka/bin/kafka-server-stop.sh
Restart=on-failure
RestartSec=5
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

Format Kafka storage (sinh cluster-id mới)

```
[ec2-user@ip-172-31-10-181 ~]$ sudo systemctl stop kafka
[ec2-user@ip-172-31-10-181 ~]$
```

Xóa dữ liệu cũ

```
[ec2-user@ip-172-31-10-181 ~]$ sudo rm -rf /usr/local/kafka/data/*
sudo rm -rf /usr/local/kafka/logs/*
```

Format storage

```
[ec2-user@ip-172-31-10-181 ~]$ /usr/local/kafka/bin/kafka-storage.sh format \
  --cluster-id $(/usr/local/kafka/bin/kafka-storage.sh random-uuid) \
  --config /usr/local/kafka/config/server.properties
Formatting metadata directory /usr/local/kafka/data with metadata.version 4.1-IV1.
[ec2-user@ip-172-31-10-181 ~]$
```

Khởi động Kafka

```
● kafka.service - Apache Kafka Server
   Loaded: loaded (/etc/systemd/system/kafka.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-01-20 17:43:22 UTC; 21s ago
     Main PID: 962239 (java)
       Tasks: 105 (limit: 1060)
      Memory: 289.5M
         CPU: 7.105s
      CGroup: /system.slice/kafka.service
              └─962239 java -Xms128M -Xmx256M -server -XX:+UseG1GC -XX:MaxGCPaus
```

Kiểm tra port

```
ec2-user@ip-172-31-10-181 ~]$ sudo ss -tulnp | grep 9092
cp    LISTEN 0      50                  *:9092
    users: (("java",pid=962239,fd=165))
ec2-user@ip-172-31-10-181 ~]$ sudo ss -tulnp | grep 9093
cp    LISTEN 0      50                  *:9093
    users: (("java",pid=962239,fd=135))
ec2-user@ip-172-31-10-181 ~]$
```

Tạo và kiểm tra topic thử

Tạo topic

```
[ec2-user@ip-172-31-10-181 local]$ /usr/local/kafka/bin/kafka-topics.sh --create \
--bootstrap-server 3.104.237.133:9092 \
--replication-factor 1 \
--partitions 1 \
--topic TopicTest
Created topic TopicTest.
```

Liệt kê topic

Producer & Consumer test

```
[ec2-user@ip-172-31-10-181 local]$ /usr/local/kafka/bin/kafka-console-producer.sh --bootstrap
-server 3.104.237.133:9092 --topic TopicTest
>hello kafka
>lam on dung loi nua
>^C[ec2-user@ip-172-31-10-181 local]$ /usr/local/kafka/bin/kafka-console-consumer.sh --bootst
rap-server 3.104.237.133:9092 --topic TopicTest
hello kafka
lam on dung loi nua
^CProcessed a total of 0 messages
[ec2-user@ip-172-31-10-181 local]$
```

CỦA THẦY

```
record = {
    "timestamp": timestamp,
    "bitcoin_usd": data["bitcoin"]["usd"],
    "ethereum_usd": data["ethereum"]["usd"]
}

df = pd.DataFrame([record])
csv_buffer = BytesIO()
df.to_csv(csv_buffer, index=False)
csv_buffer.seek(0)

filename = f"crypto_prices/{datetime.now(timezone.utc).strftime('%Y-%m-%d')}.csv"

# Append-like
try:
    existing = minio_client.get_object(bucket, filename)
    old_df = pd.read_csv(existing)
    combined_df = pd.concat([old_df, df])
    csv_buffer = BytesIO()
    combined_df.to_csv(csv_buffer, index=False)
    csv_buffer.seek(0)
except:
    pass # File chưa có -> tạo mới

minio_client.put_object(
    bucket,
    filename,
    data=csv_buffer,
    length=csv_buffer.getbuffer().nbytes,
    content_type="text/csv"
)

print(f"[{timestamp}] Saved: {record}")

except Exception as e:
    print("Error:", e)

time.sleep(15)
```

1m 39.6s

2026-01-22 15:07:08] Saved: {'timestamp': '2026-01-22 15:07:08', 'bitcoin_usd': 89167, 'ethereum_usd': 2947.21}

2026-01-22 15:07:24] Saved: {'timestamp': '2026-01-22 15:07:24', 'bitcoin_usd': 89167, 'ethereum_usd': 2947.21}

PHÁT TRIỂN THÊM

CoinGecko API -> Producer -> Kafka Topic

1. Tạo kafka topic “crypto-prices”

```
[ec2-user@ip-172-31-10-181 ~]$ /usr/local/kafka/bin/kafka-topics.sh --create \
--bootstrap-server 3.104.237.133:9092 \
--replication-factor 1 \
--partitions 1 \
--topic crypto-prices
Created topic crypto-prices.
[ec2-user@ip-172-31-10-181 ~]$
```

2. Khởi tạo Kafka Producer

```
import json
import time
import requests # Giả sử bạn dùng requests để lấy giá
from kafka import KafkaProducer
from datetime import datetime, timezone
```

✓ 0.0s

```
# 1. Khởi tạo Kafka Producer
producer = KafkaProducer(
    bootstrap_servers=['3.104.237.133:9092'],
    value_serializer=lambda x: json.dumps(x).encode('utf-8')
)

print("Đã kết nối tới Kafka Broker!")
```

✓ 0.3s

Đã kết nối tới Kafka Broker!

3. Polling vào Kafka

```
def fetch_prices():
    url = "https://api.coingecko.com/api/v3/simple/price"
    params = {"ids": "bitcoin,ethereum", "vs_currencies": "usd"}
    r = requests.get(url, params=params, timeout=10)
    if r.status_code == 200:
        return r.json()
    return {}

MAX_ROUNDS = 45
round_count = 0

while round_count < MAX_ROUNDS:
    data = fetch_prices()
    if "bitcoin" in data and "ethereum" in data:
        record = {
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "bitcoin_usd": data["bitcoin"]["usd"],
            "ethereum_usd": data["ethereum"]["usd"]
        }
        producer.send("crypto-prices", record)
        print("Sent:", record)
        round_count += 1
        time.sleep(15)
```

```
Sent: {'timestamp': '2026-01-22T13:54:21.868191+00:00', 'bitcoin_usd': 89531, 'ethereum_usd': 2968.01}
Sent: {'timestamp': '2026-01-22T13:54:37.904710+00:00', 'bitcoin_usd': 89483, 'ethereum_usd': 2964.88}
Sent: {'timestamp': '2026-01-22T13:54:53.271928+00:00', 'bitcoin_usd': 89483, 'ethereum_usd': 2964.88}
Sent: {'timestamp': '2026-01-22T13:55:08.741406+00:00', 'bitcoin_usd': 89483, 'ethereum_usd': 2964.88}
Sent: {'timestamp': '2026-01-22T13:55:24.121376+00:00', 'bitcoin_usd': 89483, 'ethereum_usd': 2964.88}
Sent: {'timestamp': '2026-01-22T13:56:42.039886+00:00', 'bitcoin_usd': 89487, 'ethereum_usd': 2968.03}
Sent: {'timestamp': '2026-01-22T13:56:57.442328+00:00', 'bitcoin_usd': 89487, 'ethereum_usd': 2968.03}
Sent: {'timestamp': '2026-01-22T13:57:12.832130+00:00', 'bitcoin_usd': 89487, 'ethereum_usd': 2968.03}
Sent: {'timestamp': '2026-01-22T13:57:29.178445+00:00', 'bitcoin_usd': 89487, 'ethereum_usd': 2968.03}
Sent: {'timestamp': '2026-01-22T13:57:44.754982+00:00', 'bitcoin_usd': 89524, 'ethereum_usd': 2969.36}
Sent: {'timestamp': '2026-01-22T13:58:00.176536+00:00', 'bitcoin_usd': 89524, 'ethereum_usd': 2969.36}
Sent: {'timestamp': '2026-01-22T13:59:17.124349+00:00', 'bitcoin_usd': 89551, 'ethereum_usd': 2969.89}
```

Kafka Topic -> MinIO bucket

1. Kết nối MinIO và Tạo bucket "crypto-data"

```
from minio.error import S3Error
import time
from minio import Minio
minio_client = Minio(
    "3.104.237.133:9000", # Endpoint
    access_key="minioadmin",
    secret_key="minioadmin",
    secure=False
)
bucket = "crypto-data"

if not minio_client.bucket_exists(bucket):
    minio_client.make_bucket(bucket)

while True:
    try:
        if not minio_client.bucket_exists(bucket):
            minio_client.make_bucket(bucket)
        print("✅ Kết nối MinIO thành công")
        break # thoát vòng lặp nếu OK

    except S3Error as e:
        print("❌ Lỗi S3 (bucket / permission):", e)
        time.sleep(30)

    except Exception as e:
        print("❌ Không kết nối được MinIO (network/timeout):", e)
        time.sleep(60)
```

✓ 0.8s

✓ Kết nối MinIO thành công

2. Kết nối Kafka Consumer

```
consumer = KafkaConsumer(
    'crypto-prices',
    bootstrap_servers='3.104.237.133:9092',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)
```

3. Tạo Batch -> Lấy dữ liệu từ Kafka -> Gom batch & append CSV -> Ghi CSV vào bucket -> Reset batch

```

batch = []
BATCH_SIZE = 10

for message in consumer:
    batch.append(message.value)
    if len(batch) >= BATCH_SIZE:
        df = pd.DataFrame(batch)
        timestamp = datetime.now(timezone.utc).strftime("%Y-%m-%d_%H-%M-%S.%f")
        filename = f"crypto_prices/{timestamp}.csv"
        # Thử đọc file đã tồn tại
        try:
            existing = minio_client.get_object(bucket, filename)
            old_df = pd.read_csv(existing)
            existing.close()
            existing.release_conn()
            df = pd.concat([old_df, df], ignore_index=True)
        except error.S3Error as e:
            # File chưa tồn tại hoặc lỗi MinIO khác
            pass

        csv_buffer = BytesIO()
        df.to_csv(csv_buffer, index=False)
        csv_buffer.seek(0)

        minio_client.put_object(
            bucket,
            filename,
            data=csv_buffer,
            length=csv_buffer.getbuffer().nbytes,
            content_type="text/csv"
        )

        print(f'Dữ liệu {len(batch)} bản ghi vào MinIO -> {filename}')
        batch = []

```

Đã lưu 10 bản ghi vào MinIO -> crypto_prices/2026-01-22_15-11-22-773007.csv

DASHBOARD HIỂN THỊ GIÁ CRYPTO REALTIME

Bước 1. Cài đặt các thư viện cần thiết

!pip install kafka-python pandas matplotlib

Bước 2. Import các thư viện sử dụng trong chương trình

```

import json
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime
from kafka import KafkaConsumer
from IPython.display import display, clear_output
import os

```

Bước 3. Cấu hình Kafka Consumer

```

BOOTSTRAP_SERVERS = "3.104.237.133"
TOPIC_NAME = "crypto-prices"
GROUP_ID = "crypto-consumer-ui"

```

Bước 4. Khởi tạo thư mục lưu dữ liệu CSV

```

OUTPUT_DIR = "/content/crypto_prices"
os.makedirs(OUTPUT_DIR, exist_ok=True)

```

Bước 5. Khởi tạo DataFrame lưu lịch sử giá

```
history_df = pd.DataFrame(  
    columns=["timestamp", "bitcoin_usd", "ethereum_usd"]  
)
```

Bước 6. Khởi tạo Kafka Consumer

```
consumer = KafkaConsumer(  
    TOPIC_NAME,  
    bootstrap_servers=BOOTSTRAP_SERVERS,  
    auto_offset_reset="latest",  
    enable_auto_commit=True,  
    group_id=GROUP_ID,  
    value_deserializer=lambda x: json.loads(x.decode("utf-8"))  
)
```

Bước 7. Bắt đầu lắng nghe dữ liệu từ Kafka

```
print("Listening Kafka topic:", TOPIC_NAME)
```

Bước 8. Thiết lập giới hạn số message xử lý

```
MAX_MESSAGES = 20
```

```
msg_count = 0
```

Bước 9. Nhận và xử lý từng message từ Kafka

```
for message in consumer:
```

```
    clear_output(wait=True)
```

```
    data = message.value
```

```
    record = {  
        "timestamp": data["timestamp"],  
        "bitcoin_usd": data["bitcoin_usd"],  
        "ethereum_usd": data["ethereum_usd"]  
    }
```

```
    df = pd.DataFrame([record])
```

```
    history_df = pd.concat([history_df, df], ignore_index=True)
```

Bước 10. Lưu dữ liệu vào file CSV theo ngày

```
filename =  
f'{OUTPUT_DIR}/crypto_prices_{datetime.utcnow().strftime('%Y-%m-%d')}.csv'  
  
if os.path.exists(filename):  
    df.to_csv(filename, mode="a", header=False, index=False)  
else:  
    df.to_csv(filename, index=False)
```

Bước 11. Hiển thị thông tin realtime và bảng dữ liệu

```
print("Realtime Crypto Prices from Kafka")  
print("Messages received:", msg_count + 1)  
print("Last update:", record["timestamp"])  
print("-" * 40)
```

```
display(history_df.tail(10))
```

Bước 12. Chuẩn hóa dữ liệu thời gian để vẽ biểu đồ

```
plot_df = history_df.copy()  
plot_df["timestamp"] = pd.to_datetime(plot_df["timestamp"])
```

Bước 13. Vẽ biểu đồ giá Bitcoin realtime

```
plt.figure(figsize=(10, 3))  
plt.plot(plot_df["timestamp"], plot_df["bitcoin_usd"])  
plt.title("Bitcoin Price (USD)")  
plt.xlabel("Time")  
plt.ylabel("USD")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

Bước 14. Vẽ biểu đồ giá Ethereum realtime

```
plt.figure(figsize=(10, 3))
plt.plot(plot_df["timestamp"], plot_df["ethereum_usd"])
plt.title("Ethereum Price (USD)")
plt.xlabel("Time")
plt.ylabel("USD")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

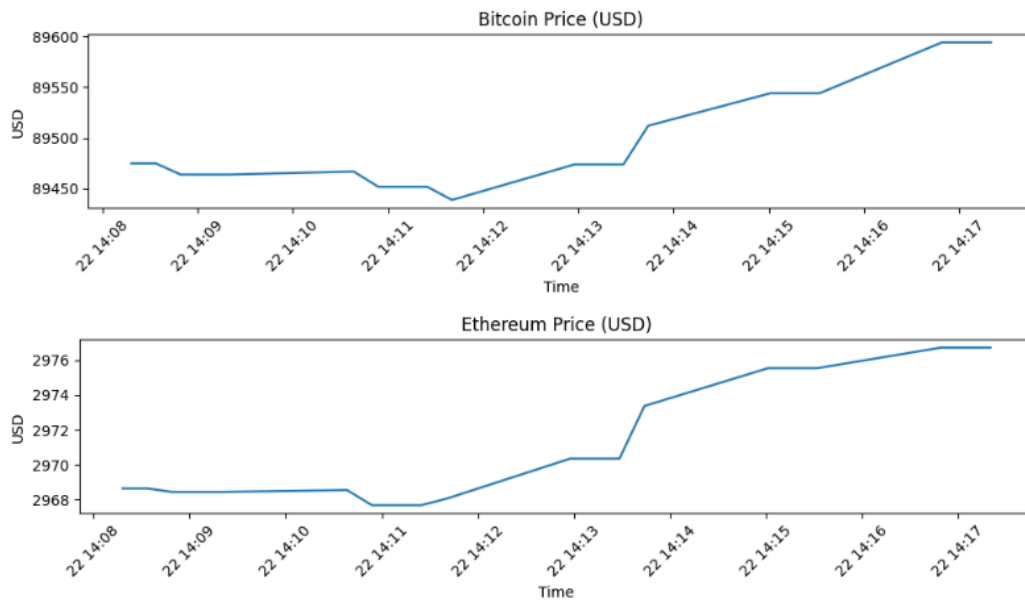
Bước 15. Kiểm soát vòng lặp và đóng Kafka Consumer

```
msg_count += 1
if msg_count >= MAX_MESSAGES:
    break

consumer.close()
```

Kết Quả:

	timestamp	bitcoin_usd	ethereum_usd
10	2026-01-22T14:12:57.623969+00:00	89474	2970.35
11	2026-01-22T14:13:13.011185+00:00	89474	2970.35
12	2026-01-22T14:13:28.399520+00:00	89474	2970.35
13	2026-01-22T14:13:44.018058+00:00	89512	2973.37
14	2026-01-22T14:15:01.182984+00:00	89544	2975.54
15	2026-01-22T14:15:16.575072+00:00	89544	2975.54
16	2026-01-22T14:15:32.091244+00:00	89544	2975.54
17	2026-01-22T14:16:49.238060+00:00	89594	2976.72
18	2026-01-22T14:17:04.639814+00:00	89594	2976.72
19	2026-01-22T14:17:20.067938+00:00	89594	2976.72



Link Google Colab:

https://colab.research.google.com/drive/1HDu0PMD2m8wV4gFh_dpUs2Oq31oxaG6n?usp=sharing